

# TP6

## Mise en place du TP

1/

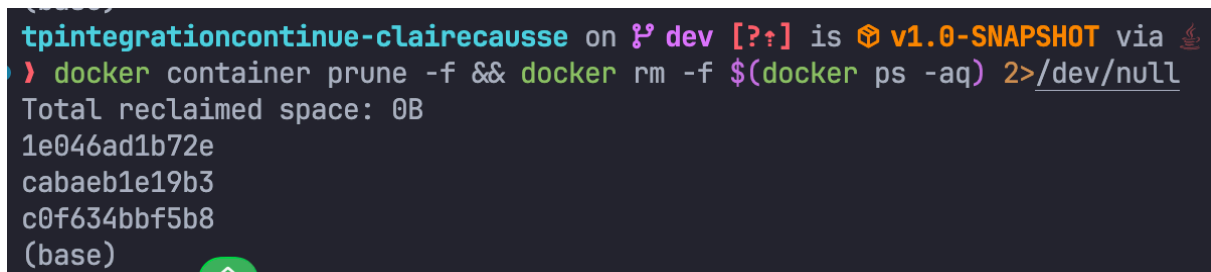
**Commande :**

```
sudo install yamllint
```

Cette commande installe **yamllint**, un outil qui permet de vérifier la syntaxe et la qualité de vos fichiers YAML. Une fois installé, on peut contrôler les fichiers avec `yamllint monfichier.yaml`.

2/

**Commande :** `docker container prune -f && docker rm -f $(docker ps -aq) 2>/dev/null`



```
tpintegrationcontinue-clairecausse on ? dev [?] is v1.0-SNAPSHOT via   
$ docker container prune -f && docker rm -f $(docker ps -aq) 2>/dev/null  
Total reclaimed space: 0B  
1e046ad1b72e  
cabaeb1e19b3  
c0f634bbf5b8  
(base)
```

Cette commande permet de supprimer en une fois tous les conteneurs, qu'ils soient arrêtés ou en cours d'exécution. On utilise `docker ps -aq` pour récupérer tous les IDs, et `docker rm -f` pour les forcer à disparaître. `docker container prune -f` élimine aussi les conteneurs déjà stoppés.

3/

**Commande :** `docker network create lemp_network --driver bridge`

```
tpintegrationcontinue-clairecausse on ⓘ dev [?⌘] is 📦 v1.0-SNAPSHOT via
• > docker network create lemp_network --driver bridge
6592e4aa63fc84c26a0fab7b2b9e7361907cbde71718812e902e222716dca965
(base)
tpintegrationcontinue-clairecausse on ⓘ dev [?⌘] is 📦 v1.0-SNAPSHOT via
```

On recrée ici un réseau Docker nommé `lemp_network` avec le driver `bridge`, ce qui correspond aux caractéristiques attendues pour relier les services du stack LEMP comme dans le TP précédent.

4/

**Commande :** `mkdir TPCompose`

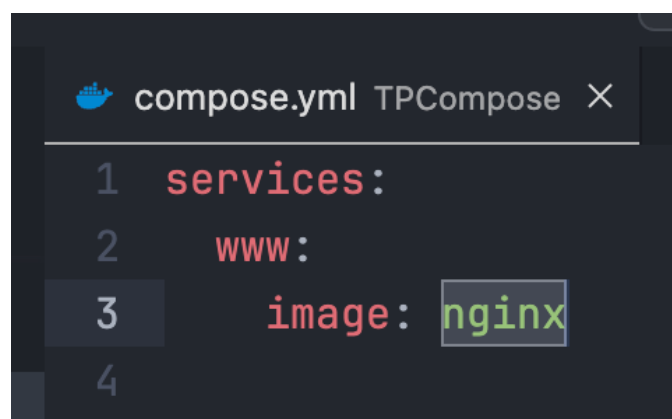
## Partie 1 – Premiers essais

1/

**Commande :** `touch compose.yml`

On crée un fichier nommé `compose.yml` dans le répertoire du TP, qui servira de base à la configuration Docker Compose. Le fichier est vide pour le moment, prêt à être rempli ensuite.

2/



```
compose.yml TPCompose X
1  services:
2    www:
3      image: nginx
4
```

On ajoute un service `www` utilisant l'image `nginx`. L'espace volontaire à la fin de la ligne `image: nginx` permet de provoquer une erreur que `yamllint` détectera

ensuite, afin de tester le linting YAML.

4/

**Commande :** `yamllint compose.yml`

On valide la syntaxe du fichier. L'espace ajouté après `nginx` provoque une erreur signalée par *yamllint*.

```
(base)
tpintegrationcontinue-clairecausse/TPCompose on ? dev [?+]
> yamllint compose.yml
compose.yml
  1:1      warning  missing document start "---" (document-start)
(base)
tpintegrationcontinue-clairecausse/TPCompose on ? dev [?+]
```



```
compose.yml TPCompose X
1  ---
2  services:
3    www:
4      image: nginx
```

**Commande :** `yamllint compose.yml`

```
(base)
tpintegrationcontinue-clairecausse/TPCompose on ? dev [?+]
● > yamllint compose.yml
(base)
tpintegrationcontinue-clairecausse/TPCompose on ? dev [?+]
```

On vérifie à nouveau que la syntaxe est correcte une fois l'espace supprimé.

5/

**Commande :** `docker compose config`

```

tpintegrationcontinue-clairecausse/TPCompose on
• > docker compose config
name: tpcompose
services:
  www:
    image: nginx
    networks:
      default: null
networks:
  default:
    name: tpcompose_default
(base)
tpintegrationcontinue-clairecausse/TPCompose on

```

On demande à Docker Compose de valider et d'afficher la configuration finale.

Docker crée automatiquement un **nom de projet** basé sur le **nom du dossier courant** : ici, il devrait générer quelque chose comme **tpcompose** (ou le nom exact de ton répertoire).

6/

**Commande :** `mkdir ServeurWeb && mv compose.yml ServeurWeb/ && cd ServeurWeb`

On crée un répertoire nommé `ServeurWeb`, on y déplace le fichier `compose.yml`, puis on se place directement dans ce nouveau dossier pour continuer la configuration du projet.

7/

**Commande :** `docker compose config`

```

tpintegrationcontinue-clairecausse/TPCompose on
• > docker compose config
name: serveurweb
services:
  www:
    image: nginx
    networks:
      default: null
networks:
  default:
    name: serveurweb_default
(base)
tpintegrationcontinue-clairecausse/TPCompose on

```

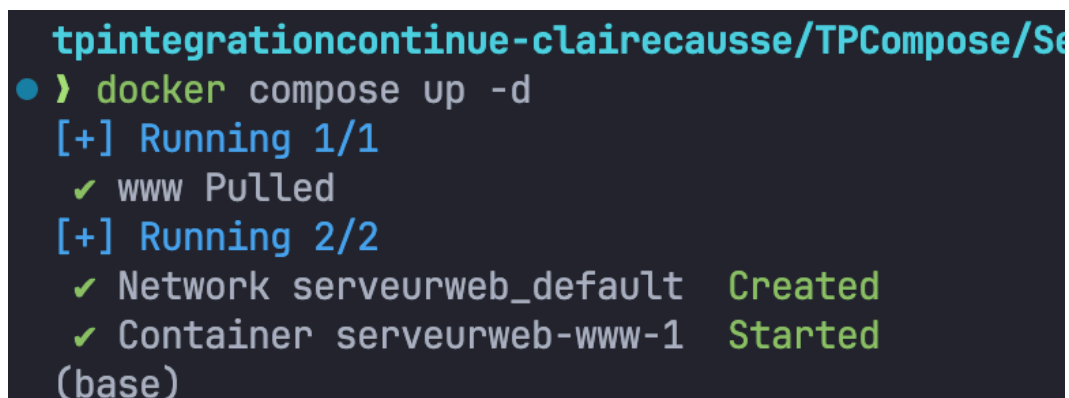
On teste à nouveau la configuration depuis le dossier `ServeurWeb`.

Docker Compose génère cette fois un **nom de projet basé sur le répertoire courant**, donc ici : `serveurweb`.

On constate aussi que Docker crée automatiquement un **réseau par défaut** nommé `serveurweb_default`, et qu'une clé `networks:` apparaît dans la configuration générée, même si elle n'était pas présente dans le fichier d'origine.

8/

**Commande :** `docker compose up -d`



```
tpintegrationcontinue-clairecausse/TPCompose/Se
● > docker compose up -d
[+] Running 1/1
  ✓ www Pulled
[+] Running 2/2
  ✓ Network serveurweb_default Created
  ✓ Container serveurweb-www-1 Started
(base)
```

On lance la création du conteneur en mode détaché.

Le conteneur est lancé **en arrière-plan**, avec un nom construit à partir du projet et du service, sous la forme :

`serveurweb-www-1`

Il est créé automatiquement, puis démarré immédiatement par Docker Compose.

9/

**Commande :** `docker ps` `docker network ls`

On constate que le conteneur `serveurweb-www-1` est bien créé, ainsi que le réseau `serveurweb_default`.

Le type du réseau est **bridge**, car Docker Compose crée par défaut un réseau en bridge.

```
tpintegrationcontinue-clairecausse/TPCompose/ServeurWeb on 8 dev [?] took 3s
• ) docker ps && docker network ls
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
831fd162f468   nginx    "/docker-entrypoint..." About a minute ago Up About a minute 80/tcp       serveurweb-www-1
NETWORK ID     NAME      DRIVER    SCOPE
582229382e77   PAS-net   bridge    local
ab37c889c7ad   PMIS-net  bridge    local
bd8ceba251d5   bridge    bridge    local
cd12cd4680f6   host      host      local
6592e4aa63fc   lemp_network bridge    local
0f214a392f64   new_bridge bridge    local
7de461fda6fe   none      null      local
4b1db0b20331   sae-s5a01-2024-sujet03_default bridge    local
1d5a3853d6d5   serveurweb_default bridge    local
3aa492640492   www      bridge    local
(base)
```

**Commande :** `docker inspect serveurweb-www-1`

On vérifie dans la section `Networks` que le conteneur est bien attaché au réseau `serveurweb_default`.

```
"Networks": {
  "serveurweb_default": {
    "IPAMConfig": null,
    "Links": null,
    "Aliases": [
      "serveurweb-www-1",
      "www"
    ],
    "NetworkID": "cd12cd4680f6",
    "EndpointID": "cd12cd4680f6",
    "Gateway": "cd12cd4680f6",
    "IPAddress": "cd12cd4680f6",
    "MacAddress": "cd12cd4680f6"
  }
}
```

On ajoute un port pour le localhost

```
compose.yml TPCompose
1 ---
2 services:
3   www:
4     image: nginx
5     ports:
6     - "80:80"
```

```

tpintegrationcontinue-clairecausse/TPCo
• > docker compose up -d
[+] Running 1/1
✓ Container serveurweb-www-1 Started
(base)
tpintegrationcontinue-clairecausse/TPCo

```

On relance un `compose up` après les changements.

On interroge Nginx depuis la machine locale, ce qui renvoie la page index par défaut fournie par l'image Nginx.

**Commande :** `curl localhost`

```

tpintegrationcontinue-clairecausse/TPCompose/ServeurWeb on 19 dev [?]
• > curl localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
(base)

```

10/

**Commande :** `docker compose down`

On arrête le service et on supprime tout ce que Compose a créé pour ce projet :

le conteneur disparaît immédiatement, et le réseau `serveurweb_default` est supprimé aussi.

On confirme ainsi que l'arrêt complet du projet Compose entraîne la suppression du conteneur et du réseau associés.

11/

**Commande :** `docker compose up -d`

On relance le conteneur en mode détaché, ce qui signifie que Docker démarre le service en arrière-plan sans occuper le terminal. Le conteneur apparaît à nouveau dans `docker ps` une fois lancé.

12/

J'ai ajouté la ligne auparavant.

**Commande :** `docker compose config`

On vérifie que la configuration générée inclut bien la clé `ports` et qu'elle est correctement formatée.

```
tpintegrationcontinue-clairecauss
● > docker compose config
name: serveurweb
services:
  www:
    image: nginx
    networks:
      default: null
    ports:
      - mode: ingress
        target: 80
        published: "80"
        protocol: tcp
    networks:
      default:
        name: serveurweb_default
(base)
tpintegrationcontinue-clairecauss
```

**Commande :** `docker compose up -d`



```
tpintegrationcontinue-clairecausse/TPCom
● > docker compose up -d
[+] Running 1/1
✓ Container serveurweb-www-1 Running
(base)
tpintegrationcontinue-clairecausse/TPCom
```

On met à jour le service **sans l'arrêter** au préalable.

Compose détecte un changement dans la configuration et **recrée automatiquement le conteneur**, tout en le remplaçant proprement. On constate donc que le conteneur est redémarré avec la nouvelle configuration, mais le projet ne s'arrête jamais complètement.

13/

**Commande :** `curl localhost`

On teste depuis la VM : Nginx répond maintenant car le port 80 du conteneur est exposé vers le port 80 de la VM. On obtient la page d'accueil par défaut de Nginx.

Ensuite, on ouvre un navigateur **sur la machine hôte** et on accède à :

`http://ipdelavm`

Comme le port 80 est exposé et accessible, le navigateur affiche lui aussi la page Nginx servie par le conteneur qui tourne dans la VM.

14/



```
1  ---
2  services:
3    www:
4      image: nginx
5      ports:
6        - "80:80"
7  networks:
8    - temp_lem_p_network
9
```

On déclare un réseau nommé `temp_lem_p_network` et on attache le service `www` dessus.

On vérifie ensuite la configuration générée pour s'assurer que le réseau est bien pris en compte :

---

**Commande :** `docker compose config`

On confirme que le service utilise maintenant `temp_lem_p_network` et que Docker ne créera plus `serveurweb_default`.

---

**Commande :** `docker compose up -d`

On relance la création avec le nouveau réseau.

Le conteneur est re-créé, cette fois attaché à `temp_lem_p_network`.

---

**Commande :** `docker inspect serveurweb-www-1`

On récupère dans la section "Networks" l'adresse IP attribuée au conteneur dans `temp_lem_p_network`.

```
"Gateway": "172.24.0.1",  
"IPAddress": "172.24.0.2",  
"IPPrefixlen": 16
```

**Commande :** `docker network ls`

On liste tous les réseaux Docker existants.

On retrouve :

- `bridge` (réseau par défaut)
- `host`
- `none`
- `temp_lemp_network`

```
tpintegrationcontinue-clairecausse/TPCompose/ServeurWeb on 8 dev [?]  
● > docker network ls  
NETWORK ID          NAME                DRIVER             SCOPE  
582229382e77        PAS-net            bridge             local  
ab37c889c7ad        PMIS-net           bridge             local  
bd8ceba251d5        bridge            bridge             local  
cd12cd4680f6        host              host              local  
6592e4aa63fc        lemp_network       bridge             local  
0f214a392f64        new_bridge         bridge             local  
7de461fda6fe        none              null              local  
4b1db0b20331        sae-s5a01-2024-sujet03_default bridge             local  
927a6d7c770d        serveurweb_default bridge             local  
14bfef1fd668        serveurweb_temp_lemp_network bridge             local  
3aa492640492        www               bridge             local  
(base)  
tpintegrationcontinue-clairecausse/TPCompose/ServeurWeb on 8 dev [?]
```

On constate donc que le conteneur a maintenant une IP différente car il est connecté au nouveau réseau user-defined `temp_lemp_network`.

15/

```

1  services:
2      www:
3          image: nginx
4          ports:
5              - "80:80"
6          networks:
7              - lemp_network
8
9  networks:
10     lemp_network:
11         external: true
12

```

On modifie la configuration pour connecter le conteneur au réseau **externe** `lemp_network` (déjà créé plus tôt).

La clé `external: true` indique à Docker Compose qu'il ne doit pas créer le réseau, mais utiliser celui qui existe déjà.

---

**Commande :** `docker compose config`

On vérifie que le service `www` utilise maintenant `lemp_network` et que Compose reconnaît qu'il s'agit d'un réseau externe.

---

**Commande :** `docker compose up -d`

On relance la création du conteneur.

Comme le réseau a changé, Compose recrée automatiquement le conteneur et l'attache à `lemp_network`.

---

**Commande :** `docker inspect serveurweb-www-1`

On vérifie dans la section "Networks" que le conteneur est bien connecté à `lemp_network`.

```

MacAddress : ,
"Networks": {
  "lemp_network": {
    "IPAMConfig": null,
    "Links": null,
    "Aliases": [
      "serveurweb-www-1",
      "www"
    ]
  }
}

```

```

"IPAddress": "172.19.0.2",

```

On retrouve l'IP associée à ce réseau, généralement dans la plage .

16/

**Commande :** `docker compose ps`

```

tpintegrationcontinue-clairecausse/TPCompose/ServeurWeb on P dev [?+]
) docker compose ps
NAME          IMAGE    COMMAND                  SERVICE    CREATED          STATUS            PORTS
serveurweb-www-1 nginx    "/docker-entrypoint..." www        About a minute ago Up About a minute 0.0.0.0:80->80/tcp, [::]:80->80/tcp
(base)
tpintegrationcontinue-clairecausse/TPCompose/ServeurWeb on P dev [?+]

```

On liste les services gérés par Docker Compose dans le projet courant.

La commande affiche le service `www`, son état (Up), son nom complet (`serveurweb-www-1`) et les ports exposés.

17/

**Commande :** `docker compose ps -a`

```

tpintegrationcontinue-clairecausse/TPCompose/ServeurWeb on P dev [?+]
) docker compose ps -a
NAME          IMAGE    COMMAND                  SERVICE    CREATED          STATUS            PORTS
serveurweb-www-1 nginx    "/docker-entrypoint..." www        About a minute ago Up About a minute 0.0.0.0:80->80/tcp, [::]:80->80/tcp
(base)

```

On liste toutes les informations des conteneurs appartenant aux services du projet : nom complet, état, ports, et configuration de chaque conteneur géré par Docker Compose.

18/

Commande : `docker compose down`

```
tpintegrationcontinue-clairecausse/TPCompos
• > docker compose down
[+] Running 1/1
✓ Container serveurweb-www-1 Removed
(base)
```

On arrête le service et on supprime le conteneur ainsi que le réseau associé au projet.

## Partie 2 – Définition d'un réseau

1/

```
compose.yml TPCompose  compose.yml S
1 networks:
2   lemp_compose:
3     driver: bridge
4     ipam:
5       config:
6         - subnet: 172.20.0.0/16
7           gateway: 172.20.0.1
8
```

On définit un réseau `lemp_compose` dans le fichier `compose.yml`.

On configure l'IPAM (IP Address Management) avec **le même subnet et la même gateway** que le réseau supprimé précédemment (`lemp_network`), comme

demandé dans le TP.

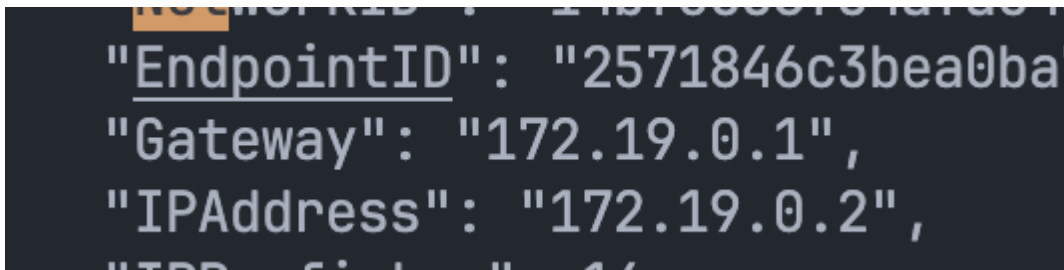
Il faut remplacer `172.20.0.0/16` et `172.20.0.1` par les valeurs exactes utilisées dans **ton TP précédent** si elles sont différentes.

2/

**Commande :** `docker compose up -d`

On lance la création du service avec le réseau `lemp_compose` récemment configuré. Docker crée le conteneur et l'attache au réseau personnalisé.

**Commande :** `docker inspect serveurweb-www-1`



```
"EndpointID": "2571846c3bea0ba"  
"Gateway": "172.19.0.1",  
"IPAddress": "172.19.0.2",  
"IPAM": {"Driver": "default", "Options": {"IPv4": "172.19.0.2", "IPv6": "fd00::2"}}
```

On consulte la section `"Networks"` pour récupérer l'IP du conteneur dans `lemp_compose`.

Elle correspond à une adresse du sous-réseau défini dans l'IPAM, du type :

`172.20.0.2`

L'IP exacte est indiquée à la ligne `"IPAddress"`.

3/



```
compose.yml ServeurWeb X compose.yml TPCompose
1  services:
2    www:
3      image: nginx
4      ports:
5        - "80:80"
6      networks:
7        lemp_compose:
8          ipv4_address: 10.20.30.5
9
10   networks:
11     lemp_compose:
12       driver: bridge
13       ipam:
14         config:
15           - subnet: 10.20.30.0/24
16
```

On attribue une **IP fixe** au conteneur grâce au champ `ipv4_address` .

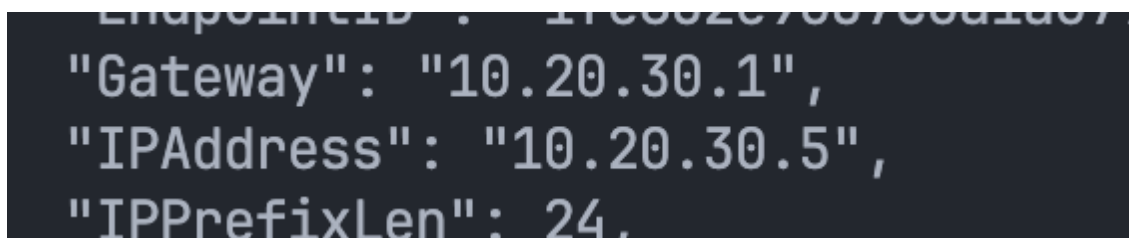
Le subnet doit être cohérent avec l'adresse choisie : ici `10.20.30.0/24` permet d'utiliser `10.20.30.5` .

Ensuite on relance :

**Commande :** `docker compose up -d`

On inspecte le conteneur pour vérifier l'IP attribuée :

**Commande :** `docker inspect serveurweb-www-1`

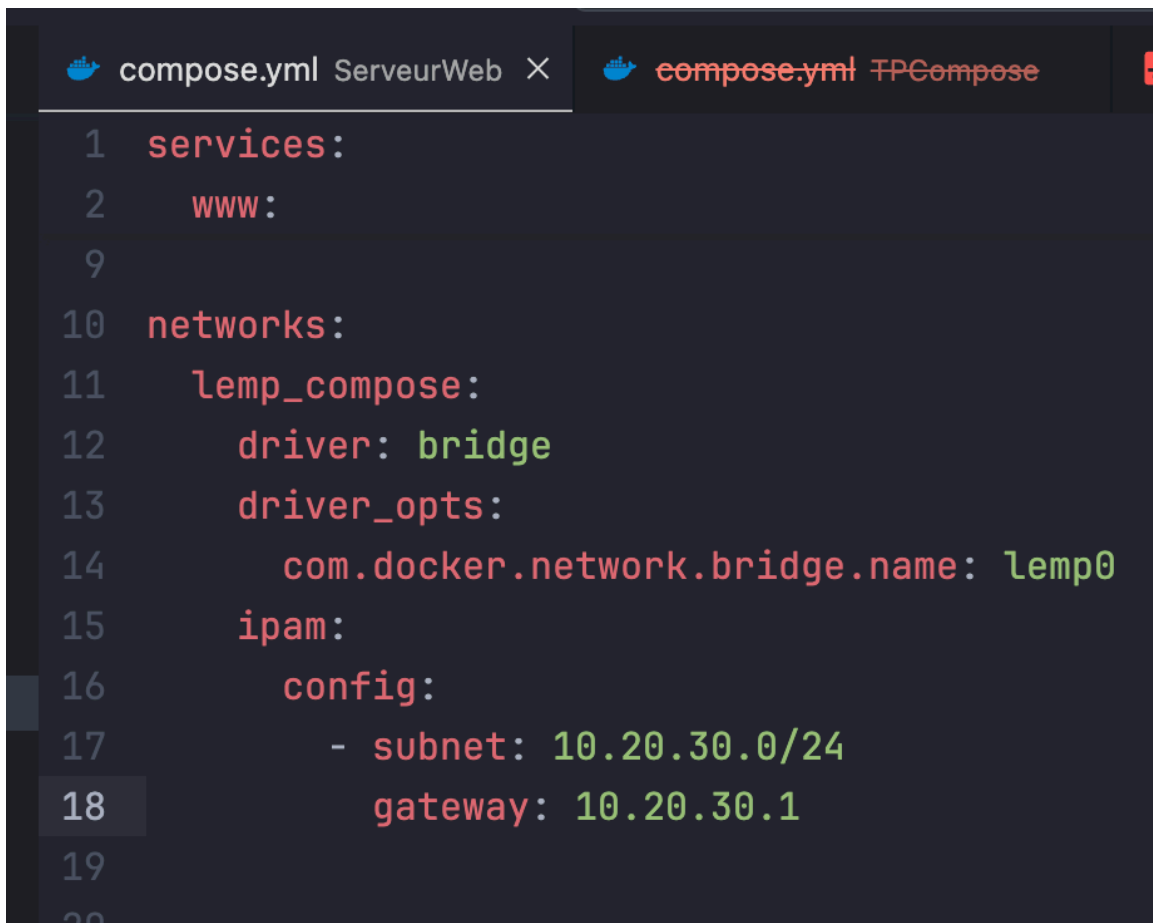


```
EndpointID : 1f00207007001d07
"Gateway": "10.20.30.1",
"IPAddress": "10.20.30.5",
"IPPrefixLen": 24,
```

On confirme que l'adresse `"IPAddress"` du conteneur est bien `10.20.30.5` .



4/



```
1  services:
2    www:
9
10  networks:
11    lemp_compose:
12      driver: bridge
13      driver_opts:
14        com.docker.network.bridge.name: lemp0
15      ipam:
16        config:
17          - subnet: 10.20.30.0/24
18            gateway: 10.20.30.1
19
20
```

On ajoute `driver_opts` avec l'option `com.docker.network.bridge.name` pour que le réseau créé apparaisse clairement dans les interfaces réseau de la VM sous le nom **lemp0** (au lieu d'un nom généré automatiquement comme `br-39abcf...`).

Ensuite on relance :

**Commande :** `docker compose up -d`

Puis on vérifie dans les interfaces de la VM :

**Commande :** `ifconfig`

On constate alors la présence de l'interface réseau nommée **lemp0**, associée au réseau Docker `lemp_compose`.

## Partie 3 – Définition de volumes

1/

**Commande :** `mkdir html nginx-conf`

On crée les deux sous-répertoires demandés : `html` pour le contenu web et `nginx-conf` pour la configuration Nginx.

On copie dans `html/` un ancien fichier `index.html` déjà utilisé précédemment.

**Commande :** `touch nginx-conf/nginx.conf`

On crée le fichier de configuration Nginx.

Dans ce fichier, on adapte le contenu de `default.conf` du TP précédent, mais en commentant toutes les lignes liées à FastCGI.

Exemple minimal :

```
server {
    listen    80;
    server_name localhost;

    root /var/www/html;
    index index.html;

    #location ~ \.php$ {
    #    include fastcgi_params;
    #    fastcgi_pass php:9000;
    #    fastcgi_index index.php;
    #    fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
    #}
}
```

On gardera uniquement l'essentiel pour servir du HTML simple.

2/

```
compose.yml ServeurWeb  nginx.conf nginx-conf X  compose.yml TPCompose
1  services:
2    www:
3      image: nginx
4      ports:
5        - "80:80"
6      volumes:
7        - ./html:/usr/share/nginx/html:ro
8        - ./nginx-conf/nginx.conf:/etc/nginx/conf.d/default.conf:ro
9      networks:
10       - lemp_compose
11  db:
```

- le répertoire `html/` → au dossier web interne de Nginx ( `/usr/share/nginx/html` )
- le fichier `nginx.conf` → au fichier de configuration interne ( `/etc/nginx/conf.d/default.conf` )

Les deux montages sont en lecture seule ( `:ro` ) pour éviter toute modification accidentelle dans le conteneur.

**Commande :** `docker compose config`

On vérifie que les volumes sont bien pris en compte dans la configuration générée par Docker Compose.

**Commande :** `docker compose up -d`

On met à jour le service. Docker détecte la modification de volumes et recrée automatiquement le conteneur avec la nouvelle configuration Nginx.

**Commande :** `curl localhost`

On teste le service depuis la VM.

On doit obtenir le contenu du `index.html` que tu as placé dans `html/` , ce qui confirme que :

3/

**Commande :** `docker exec -it serveurweb-www-1 sh`

On entre dans le conteneur Nginx avec un shell interactif.

Cela permet d'examiner directement les fichiers internes, notamment la configuration montée depuis l'hôte.

**Commande :** `cat /etc/nginx/conf.d/default.conf`

On vérifie que le fichier de configuration Nginx dans le conteneur correspond bien à celui du répertoire `nginx-conf/nginx.conf`.

On s'assure également que les lignes FastCGI sont correctement commentées et que la directive `root` pointe vers `/usr/share/nginx/html`.

4/

**Commande :** (contenu à ajouter/modifier dans `compose.yml`)

```
services:
  www:
    image: nginx
    ports:
      - "80:80"
    volumes:
      - ./html:/usr/share/nginx/html:ro
    networks:
      - lemp_compose
```

On relie maintenant **le sous-répertoire** `html` du projet au **répertoire des pages web** du conteneur :

`/usr/share/nginx/html`.

Le montage est en lecture seule (`:ro`) car les fichiers du conteneur n'ont pas vocation à être modifiés.

Puis :

`docker compose config` `docker compose up -d` `curl localhost`

5/

```
services:
  www:
    image: nginx
```

```
ports:
  - "80:80"
volumes:
  - type: bind
    source: ./html
    target: /usr/share/nginx/html
    read_only: true
networks:
  - lemp_compose
```

```
php:
  image: php:8.3-fpm
  volumes:
    - type: bind
      source: ./php
      target: /var/www/html
      read_only: false
  networks:
    - lemp_compose
```

```
networks:
  lemp_compose:
    driver: bridge
  ipam:
    config:
      - subnet: 10.20.30.0/24
        gateway: 10.20.30.1
```

On ajoute un service **php** basé sur l'image `php:8.3-fpm` .

On lie le sous-répertoire `php/` au répertoire `/var/www/html` du conteneur PHP via la version complète de la syntaxe des volumes.

---

**Commande :** `mkdir php`

On crée le répertoire destiné aux scripts PHP.

---

**Commande :** `echo "<?php phpinfo();" > php/info.php`

On place un fichier `info.php` dans le répertoire PHP pour test.

```

server {
    listen 80;
    server_name localhost;

    root /usr/share/nginx/html;
    index index.html index.php;

    location / {
        try_files $uri $uri/ =404;
    }

    location ~ \.php$ {
        include fastcgi_params;
        fastcgi_pass php:9000;
        fastcgi_index index.php;
        fastcgi_param SCRIPT_FILENAME /var/www/html$fastcgi_script_name;
    }
}

```

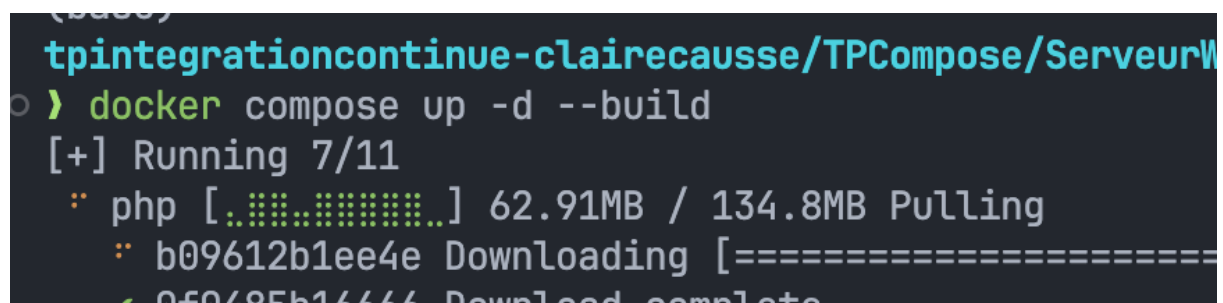
On redirige maintenant les scripts PHP vers le service `php` sur le port `9000`.

**Commande :** `docker compose config`

On vérifie que les volumes et les réseaux des deux services sont correctement définis.

**Commande :** `docker compose up -d`

On met à jour les services. Docker relance `www` et crée `php`.



```

tpintegrationcontinue-clairecausse/TPCompose/ServeurW
> docker compose up -d --build
[+] Running 7/11
  ⋄ php [.....] 62.91MB / 134.8MB Pulling
  ⋄ b09612b1ee4e Downloading [=====]
  ⋄ 050485b16666 Download complete

```

**Commande :** `curl localhost/info.php`

On teste : la page doit afficher la sortie de `phpinfo()` , ce qui confirme que :

6/

**Commande :** `docker volume ls`

On liste l'ensemble des volumes Docker présents sur la VM.

On constate que **tous les volumes utilisés dans le projet sont des volumes de type bind-mount**, car ils proviennent directement des sous-répertoires du dossier `ServeurWeb` ( `html/` , `php/` , `nginx-conf/` ).

7/

**Commande :** `docker inspect serveurweb-www-1`

On inspecte le conteneur Nginx pour afficher la section "**Mounts**", qui liste tous les bind-mounts utilisés.

```
    "Mounts": [
      {
        "Type": "bind",
        "Source": "/Users/clairecausse/MacBook de Claire/IUT/M1/M1/Principes et outils pour DevOps/tpintegrationcontinue-clairecausse/TPCompose/ServeurWeb/html",
        "Destination": "/usr/share/nginx/html",
        "Mode": "rw",
        "RW": true,
        "Propagation": "rprivate"
      }
    ]
  }
```

Ces bind-mounts confirment que les répertoires et fichiers locaux sont correctement liés dans le conteneur.

---

**Commande :** `docker inspect serveurweb-php-1`

On inspecte également le conteneur PHP-FPM.

```
    "Mounts": [
      {
        "Type": "bind",
        "Source": "/Users/clairecausse/MacBook de Claire/IUT/M1/M1/Principes et outils pour DevOps/tpintegrationcontinue-clairecausse/TPCompose/ServeurWeb/php",
        "Destination": "/var/www/html",
        "Mode": "rw",
        "RW": true,
        "Propagation": "rprivate"
      }
    ]
  }
```

Dans la section "**Mounts**", on constate le bind suivant :

- `./php` → `/var/www/html`

Cela confirme que le conteneur PHP accède bien aux mêmes scripts PHP que Nginx.

```

services:
  www:
    image: nginx
    ports:
      - "80:80"
    volumes:
      - ./html:/usr/share/nginx/html
      - ./php:/var/www/html
      - ./nginx-conf/nginx.conf:/etc/nginx/conf.d/default.conf
    networks:
      lemp_compose:
        ipv4_address: 10.20.30.5

  php:
    image: php:8.3-fpm
    volumes:
      - ./php:/var/www/html
    networks:
      - lemp_compose

  mariadb:
    container_name: bd
    image: mariadb:11.4.3
    environment:
      - MYSQL_ROOT_PASSWORD=root
      - MYSQL_DATABASE=tp_db
      - MYSQL_USER=user
      - MYSQL_PASSWORD=userpass
    networks:
      - lemp_compose

```

On ajoute un nouveau service nommé **mariadb**, mais en donnant au conteneur le nom explicite **bd** grâce à `container_name: bd`.

On place le service sur le même réseau `lemp_compose` afin que Nginx et PHP-FPM puissent communiquer avec la base via le nom du service `mariadb` ou directement `bd`.



Comme demandé, l'image utilisée est **mariadb:11.4.3**.

9/

```
services:
  www:
    image: nginx
    ports:
      - "80:80"
    volumes:
      - ./html:/usr/share/nginx/html
      - ./php:/var/www/html
      - ./nginx-conf/nginx.conf:/etc/nginx/conf.d/default.conf
    networks:
      lemp_compose:
        ipv4_address: 10.20.30.5

  php:
    image: php:8.3-fpm
    volumes:
      - ./php:/var/www/html
    networks:
      - lemp_compose

  mariadb:
    container_name: bd
    image: mariadb:11.4.3
    networks:
      - lemp_compose
    volumes:
      - donneesbd:/var/lib/mysql

volumes:
  donneesbd:

networks:
  lemp_compose:
```

```
driver: bridge
driver_opts:
  com.docker.network.bridge.name: lemp0
ipam:
  config:
    - subnet: 10.20.30.0/24
      gateway: 10.20.30.1
```

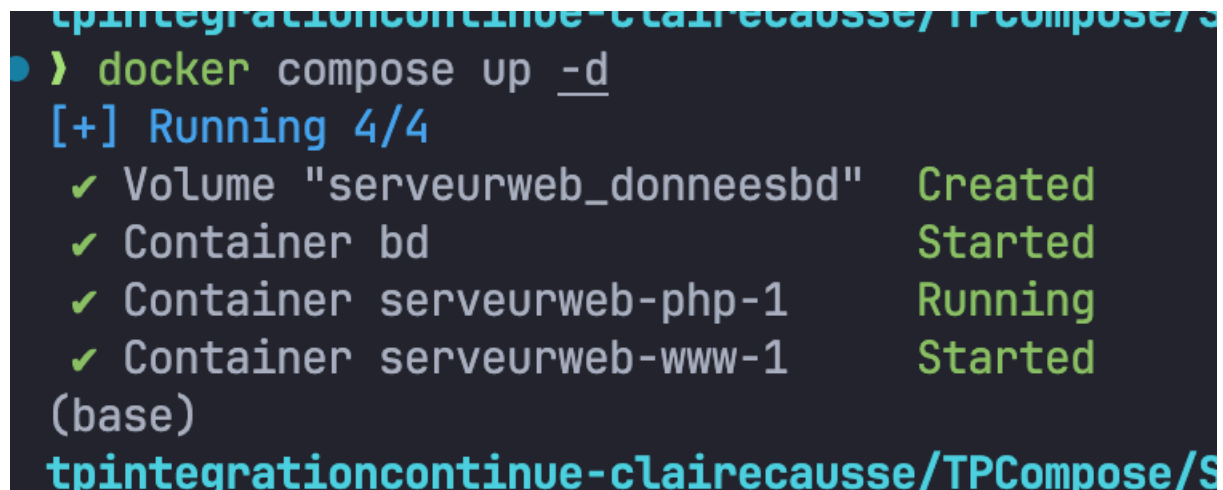
On déclare un volume nommé **donneesbd** dans la section `volumes:` du fichier Compose.

On indique ensuite que ce volume est utilisé par le service `mariadb` via le montage : `donneesbd:/var/lib/mysql`

Ce répertoire est celui où MariaDB stocke ses données internes, ce qui garantit leur persistance.

10/

**Commande :** `docker compose up -d`



```
tpintegrationcontinue-clairecausse/TPCompose/S
• > docker compose up -d
[+] Running 4/4
✓ Volume "serveurweb_donneesbd"    Created
✓ Container bd                     Started
✓ Container serveurweb-php-1       Running
✓ Container serveurweb-www-1       Started
(base)
tpintegrationcontinue-clairecausse/TPCompose/S
```

On lance tous les services, y compris le nouveau conteneur MariaDB (`bd`). Docker crée le volume `donneesbd` et attache le conteneur à ce volume.

On constate ensuite que le conteneur `bd` est bien démarré et en état *Up* via `docker compose ps`.

11/

**Commande :** `docker volume ls`

On liste les volumes Docker présents sur la VM.

Le volume que vous venez de déclarer dans `compose.yml` apparaît maintenant dans la liste sous le nom :



```
local serveurweb_donneesbd
```

`serveurweb_donneesbd`

Il est automatiquement préfixé par le **nom du projet Compose** ( `serveurweb` ) suivi du nom déclaré ( `donneesbd` ).

12/

Le volume est déjà nommé `donneesbd`

Mais si on avait du changer le nom :

```
volumes:
  donneesbd:
    name: donneesbd
```

13/

**Commande :** `docker compose down`

On arrête tous les services du projet et on libère les conteneurs associés, sans supprimer les volumes.

**Commande :** `docker volume ls`

On liste les volumes Docker.

On constate que le volume persistant **donneesbd** est toujours présent, car Docker Compose ne supprime jamais les volumes déclarés, même après un `down` .

## Partie 4 – Commandes utilitaires

1/

**Commande :** `docker compose --help`

On consulte l'ensemble des commandes disponibles pour Docker Compose.

La liste affiche toutes les actions possibles

: `up`, `down`, `ps`, `logs`, `exec`, `config`, `build`, `pull`, etc., ainsi que leur description.

2/

**Commande :** `docker compose ls`

On liste tous les projets Docker Compose actifs sur la machine.

Pour chaque projet, Docker indique :

- le nom du projet,
- le nombre de conteneurs,
- l'état,
- et surtout le chemin du fichier `compose.yml` utilisé pour créer ce projet.

3/

**Commande :** `docker compose -p serveurweb pause`

```
tpintegrationcontinue-clairecausse/TPCompose/ServeurWeb on ❸ dev [?+]
```

```
> docker compose ls
```

| NAME       | STATUS     | CONFIG FILES                       |
|------------|------------|------------------------------------|
| serveurweb | running(1) | /Users/clairecausse/MacBook de Cla |

```
sse/TPCompose/ServeurWeb/compose.yml  
(base)
```

```
tpintegrationcontinue-clairecausse/TPCompose/ServeurWeb on ❸ dev [?+]
```

```
> docker ps
```

| CONTAINER ID | IMAGE       | COMMAND                  | CREATED       | STAT |
|--------------|-------------|--------------------------|---------------|------|
| cfa491942ad5 | php:8.3-fpm | "docker-php-entrypoi..." | 7 seconds ago | Up 6 |

```
(base)
```

```
tpintegrationcontinue-clairecausse/TPCompose/ServeurWeb on ❸ dev [?+]
```

```
> docker compose -p serveurweb pause
```

```
[+] Pausing 1/1
```

```
✓ Container serveurweb-php-1 Paused
```

```
(base)
```

```
tpintegrationcontinue-clairecausse/TPCompose/ServeurWeb on ❸ dev [?+]
```

On se place dans un autre répertoire (par exemple `cd ~`), puis on met en pause les services du projet en précisant le nom du projet (`serveurweb`), puisque le fichier `compose` n'est plus dans le répertoire courant.

Les conteneurs passent alors à l'état **Paused**, ce que l'on peut vérifier avec :

**Commande :** `docker ps`

On constate dans la colonne **STATUS** que les conteneurs sont marqués **Paused**, ce qui signifie qu'ils ont bien été mis en pause.

4/

**Commande :** `docker compose -p serveurweb unpause`

```
tpintegrationcontinue-clairecausse/TPCompos
> docker compose -p serveurweb unpause
[+] Running 1/1
✓ Container serveurweb-php-1 Unpaused
(base)
tpintegrationcontinue-clairecausse/TPCompos
```

On réactive les services du projet **serveurweb** depuis n'importe quel répertoire (puisque l'on utilise `-p serveurweb`).

Les conteneurs repassent de l'état **Paused** à **Up** immédiatement.

Ensuite on vérifie :

**Commande :** `docker ps`

On constate que les services sont à nouveau actifs, et on remarque que le temps indiqué dans la colonne **STATUS** continue (ex. "Up 10 minutes"), car un `unpause` ne redémarre pas les conteneurs : il les réactive simplement en continuant leur temps d'exécution.

## Partie 5 – Environnement

1/

```
services:
  www:
    image: nginx
    ports:
      - "80:80"
    depends_on:
      - php
    volumes:
      - ./html:/usr/share/nginx/html
      - ./php:/var/www/html
      - ./nginx-conf/nginx.conf:/etc/nginx/conf.d/default.conf
    networks:
      lemp_compose:
        ipv4_address: 10.20.30.5
```

On ajoute la clé **depends\_on** au service `www`.

Cela indique à Docker Compose que le conteneur **php** doit être démarré en premier.

Docker Compose respecte cet ordre lors du `up`.

---

**Commande :** `docker compose up -d`

On met à jour les services et relance la stack.

Nginx attend maintenant que PHP-FPM soit démarré avant de démarrer son propre conteneur.

---

**Commande :** `docker ps`

On constate que les conteneurs sont démarrés dans le bon ordre et que le conteneur `www` n'apparaît pas en "Created" ou "Restarting" avant `php`.

2/

```
mariadb:
  container_name: bd
```

```
image: mariadb:11.4.3
volumes:
  - donneesbd:/var/lib/mysql
restart: on-failure
networks:
  - lemp_compose
```

```
tpintegrationcontinue-clairecausse/TPCompose/ServeurWeb on ? dev [?]
```

```
> docker ps
```

| CONTAINER ID | IMAGE       | COMMAND                  | CREATED        | STATUS        |
|--------------|-------------|--------------------------|----------------|---------------|
| 33db038a7149 | nginx       | "/docker-entrypoint..."  | 37 minutes ago | Up 37 minutes |
| 7f49efc9dc44 | php:8.3-fpm | "docker-php-entrypoi..." | 37 minutes ago | Up 37 minutes |
| (base)       |             |                          |                |               |

```
tpintegrationcontinue-clairecausse/TPCompose/ServeurWeb on ? dev [?]
```

On ajoute l'option `restart: on-failure` afin que le conteneur **bd** soit automatiquement relancé uniquement si son démarrage échoue (code d'erreur  $\neq 0$ ).

Cette stratégie ne redémarre pas le conteneur en cas d'arrêt volontaire ou normal, uniquement en cas d'échec technique.

**Commande :** `docker ps`

On liste plusieurs fois de suite les conteneurs actifs.

On constate que le conteneur **bd** n'apparaît pas forcément comme "Up" immédiatement :

cela s'explique par le fait que MariaDB peut échouer à démarrer plusieurs fois de suite (par exemple si la config est vide ou si le volume existe déjà).

Dans ce cas, Docker tente de le redémarrer en boucle, mais comme l'échec est très rapide, on peut ne jamais le voir dans la liste active au moment exact où il est "Up".

3/

**Commande :** `docker logs bd`

On consulte les journaux du conteneur **bd** (MariaDB).

On visualise les messages de démarrage, ainsi que les éventuelles erreurs expliquant pourquoi le conteneur échoue à démarrer ou redémarre automatiquement.

4/

On ajoute :

```
mariadb:
  container_name: bd
  image: mariadb:11.4.3
  volumes:
    - donneesbd:/var/lib/mysql
  environment:
    - MARIADB_ROOT_PASSWORD=mariadb
  restart: on-failure
  networks:
    - lemp_compose
```

On ajoute la variable d'environnement **MARIADB\_ROOT\_PASSWORD** avec la valeur **mariadb**, ce qui permet au serveur MariaDB de démarrer correctement en définissant un mot de passe root obligatoire.

**Commande :** `docker compose up -d`

On relance les services pour appliquer la nouvelle configuration. Le conteneur `bd` est recréé et utilise maintenant le mot de passe root spécifié.

**Commande :** `docker logs bd`

On consulte les logs du conteneur MariaDB.

```
tpintegrationcontinue-clairecausse/TPCompose/ServeurWeb on ? dev [?+]
> docker logs bd
2025-11-14 9:47:59 0 [Note] InnoDB: Removed temporary tablespace data file: "ibtmp1"
2025-11-14 9:47:59 0 [Note] InnoDB: Shutdown completed; log sequence number 47891; transaction id 15
2025-11-14 9:47:59 0 [Note] mariadbd: Shutdown complete

2025-11-14 09:47:59+00:00 [Note] [Entrypoint]: Temporary server stopped

2025-11-14 09:47:59+00:00 [Note] [Entrypoint]: MariaDB init process done. Ready for start up.

2025-11-14 9:47:59 0 [Note] Starting MariaDB 11.4.3-MariaDB-ubu2404 source revision 5ab81ffe0097a22a77
ynTc0Bw8FWiGI= as process 1
```

On constate que le service démarre correctement, sans erreur, grâce à la variable `MARIADB_ROOT_PASSWORD` qui fournit la configuration minimale nécessaire pour initialiser le serveur MariaDB.

5/



```
(base)
tpintegrationcontinue-clairecausse/TPCompose/S
> docker inspect bd | grep IPAddress
      "SecondaryIPAddresses": null,
      "IPAddress": "",
      "IPAddress": "10.20.30.3",
(base)
tpintegrationcontinue-clairecausse/TPCompose/S
```

**Commande :** `nc -zv 10.20.30.3 3306`

On utilise `nc` (netcat) pour tester la connexion TCP vers le port 3306 (port MySQL/MariaDB) du conteneur **bd**.

L'option `-z` teste sans envoyer de données, et `-v` affiche le résultat.

Si la connexion est possible, `nc` affiche un message indiquant que le port est **open**.

Si la connexion échoue, le port est indiqué comme **refused** ou **timed out**.

6/

On ajoute:

```
hostname: bd
```

Elle définit explicitement le **nom d'hôte interne** du conteneur MariaDB dans le réseau Docker.

Ce hostname devient donc résolvable comme une adresse réseau par les autres conteneurs du projet (ex. `php` peut accéder à MariaDB via `bd:3306`).

Cette configuration permet d'utiliser un nom stable, indépendant de l'IP, conformément aux bonnes pratiques Docker.

7/

**Commande :** `docker exec -it bd env`

```
(base)
tpintegrationcontinue-clairecausse/TPCompose/Se
• > docker exec -it bd env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/u
HOSTNAME=bd
TERM=xterm
MARIADB_ROOT_PASSWORD=mariadb
GOSU_VERSION=1.17
LANG=C.UTF-8
MARIADB_VERSION=1:11.4.3+maria~ubu2404
HOME=/root
```

On affiche toutes les variables d'environnement du conteneur **bd**.

On y retrouve celles définies dans le `compose.yml`, comme :

- `MARIADB_ROOT_PASSWORD=mariadb`
- `HOSTNAME=bd`
- ainsi que toutes les variables internes propres à MariaDB et à Docker.

8/

**Commande :** `export MONPASSWD=monmdp`

On définit la variable d'environnement Linux **MONPASSWD** dans ton shell courant avec la valeur `monmdp`.

```
environment:
  - MARIADB_ROOT_PASSWORD=${MONPASSWD}
```

On utilise maintenant **la valeur de la variable Linux MONPASSWD** pour remplir le mot de passe root de MariaDB.

Docker Compose remplace automatiquement `${MONPASSWD}` par la valeur exportée dans ton terminal.

**Commande :** `docker compose config`

On vérifie la configuration générée.

Dans la section du service mariadb, on doit voir :

```
MARIADB_ROOT_PASSWORD: monmdp
```

Ce qui confirme que :

- Docker Compose a bien récupéré la valeur de la variable Linux,
- et qu'il l'a substituée dans la configuration effective du projet.

Si tu veux, tu peux m'envoyer la sortie de `docker compose config`, je valide avec toi que tout est correct.

```
container_name: bd
environment:
  MARIADB_ROOT_PASSWORD: monmdp
hostname: bd
image: mariadb:11.4.3
networks:
```

9/

**Commande :** `docker exec -it serveurweb-php-1 sh`

On entre dans le conteneur PHP, qui est **sur le réseau lemp\_compose**, donc capable de joindre MariaDB.

**Commande :** `nc -zv bd 3306`

On utilise le hostname `bd` (défini dans `compose.yml`).

La commande tente d'établir une connexion TCP vers MariaDB.

On optient:

```
Connection to bd 3306 port [tcp/mysql] succeeded!
```

Cela confirme que :

- le service MariaDB écoute bien sur le port 3306
- le réseau lemp\_compose fonctionne
- le hostname bd est résolvable par les autres conteneurs
- la configuration Docker Compose est correcte

10/

On ajoute la directive :

```
hostname: bd
```

Cette entrée définit explicitement le **nom d'hôte interne** du conteneur MariaDB dans le réseau Docker.

Ainsi, les autres conteneurs (comme `php` ou `www`) peuvent accéder à MariaDB simplement via :

```
bd:3306
```

Ce hostname reste stable même si l'IP interne change, ce qui correspond aux bonnes pratiques Docker.

11/

Commande : `docker exec -it bd env`

## tpintegrationcontinue-clairecausse/TPCompo

```
> docker exec -it bd env
PATH=/usr/local/sbin:/usr/local/bin:/usr/s
HOSTNAME=bd
TERM=xterm
MARIADB_ROOT_PASSWORD=monmdp
GOSU_VERSION=1.17
LANG=C.UTF-8
MARIADB_VERSION=1:11.4.3+maria~ubu2404
HOME=/root
```

Cette commande affiche les variables d'environnement du conteneur **bd**.

On y voit bien `HOSTNAME=bd` et `MARIADB_ROOT_PASSWORD` avec la valeur définie dans `compose.yml`.

12/

Commande : `export MONPASSWD=monmdp`

On définit une variable d'environnement Linux nommée **MONPASSWD** avec la valeur **monmdp**.

Elle sera utilisable par Docker Compose.

```
donneesbd:/var/lib/mysql
environment:
  - MARIADB_ROOT_PASSWORD=${MONPASSWD}
restart: on-failure
```

On indique maintenant que le mot de passe root de MariaDB doit prendre la valeur de la variable d'environnement Linux.

Commande : `docker compose config`

Cette commande permet de vérifier que la configuration est correcte.

On doit voir dans la sortie :

```
container_name: bd
environment:
  MARIADB_ROOT_PASSWORD: monmdp
hostname: bd
```

→ Cela confirme que la variable Linux est bien injectée dans la configuration Docker Compose.

13/

Commande : `docker compose up -d`

On relance les services en arrière-plan pour appliquer la nouvelle configuration utilisant la variable d'environnement Linux.

Commande : `docker exec -it bd env`

On affiche les variables d'environnement du conteneur **bd**.

On constate que :

```
HOSTNAME=bd
TERM=xterm
MARIADB_ROOT_PASSWORD=monmdp
Docker version 1.13.1
```

→ Le conteneur utilise bien la valeur provenant de la variable d'environnement Linux **MONPASSWD**.

14/

Commande : `echo "PASSWDFIC=passwdfic" > .env`

On crée un fichier **.env** contenant la variable `PASSWDFIC` avec la valeur **passwdfic**.

Docker Compose lit automatiquement ce fichier.

```
- donneesbd:/var/lib/mysql
environment:
  - MARIADB_ROOT_PASSWORD=${PASSWDFIC}
restart: on-failure
```

On utilise maintenant la variable définie dans `.env` pour définir le mot de passe root de MariaDB.

Commande : `docker compose config`

On vérifie la configuration générée.

On doit voir :

```
container_name: bd
environment:
  MARIADB_ROOT_PASSWORD: passwdfic
hostname: bd
```

→ Cela confirme que Docker Compose utilise bien la variable du fichier `.env`.

Commande : `docker compose up -d`

On relance les services avec la nouvelle configuration.

Commande : `docker exec -it bd env`

On affiche les variables d'environnement du conteneur **bd**.

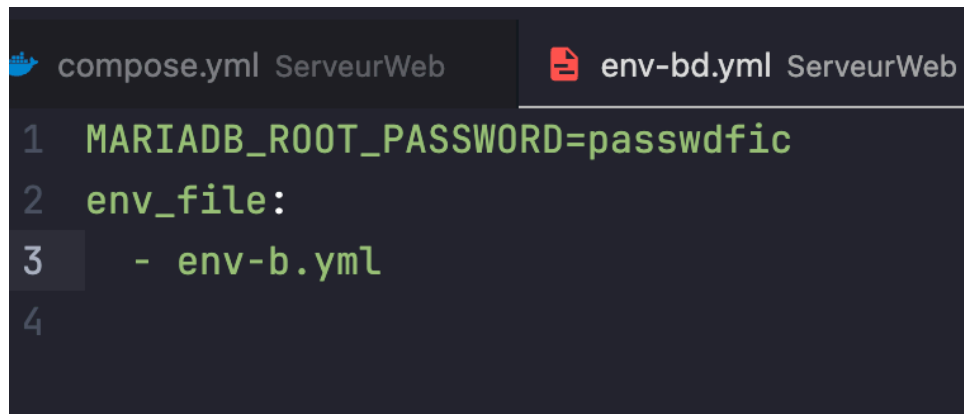
```
tpintegrationcontinue-clairecausse/
> docker exec -it bd env
PATH=/usr/local/sbin:/usr/local/bin
HOSTNAME=bd
TERM=xterm
MARIADB_ROOT_PASSWORD=passwdfic
GOSU_VERSION=1.17
LANG=C.UTF-8
```

→ La variable provenant du fichier `.env` est bien prise en compte.

15/

On crée le fichier **env-bd.yml** et on y met la variable d'environnement pour MariaDB.

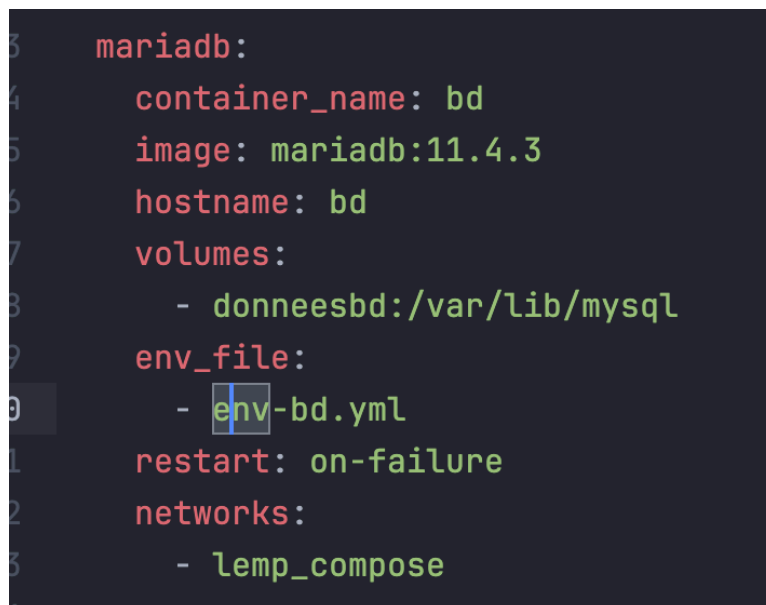
Ce fichier contiendra désormais la configuration de l'environnement de la BD.



The screenshot shows a code editor with two tabs: 'compose.yml ServeurWeb' and 'env-bd.yml ServeurWeb'. The 'env-bd.yml' tab is active and displays the following content:

```
1 MARIADB_ROOT_PASSWORD=passwdfic
2 env_file:
3   - env-bd.yml
4
```

→ Le mot de passe root est maintenant chargé depuis le fichier externe **env-bd.yml**.



The screenshot shows a code editor with a single tab displaying the configuration for the 'mariadb' service in a 'compose.yml' file. The content is as follows:

```
3 mariadb:
4   container_name: bd
5   image: mariadb:11.4.3
6   hostname: bd
7   volumes:
8     - donneesbd:/var/lib/mysql
9   env_file:
10    - env-bd.yml
11   restart: on-failure
12   networks:
13     - lemp_compose
```

Commande : `docker compose config`

On vérifie la configuration générée.

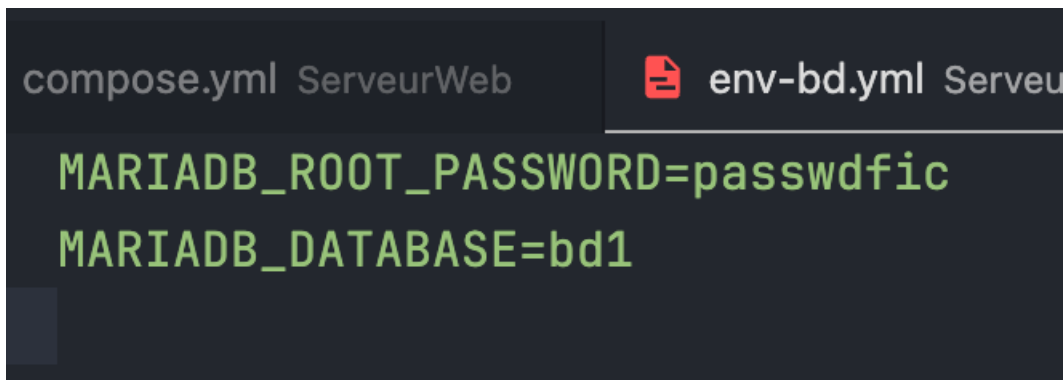


On doit constater dans la section mariadb :

```
MARIADB_ROOT_PASSWORD: passwdfic
```

→ Ce qui confirme que la variable d'environnement est bien récupérée depuis **env-bd.yml**.

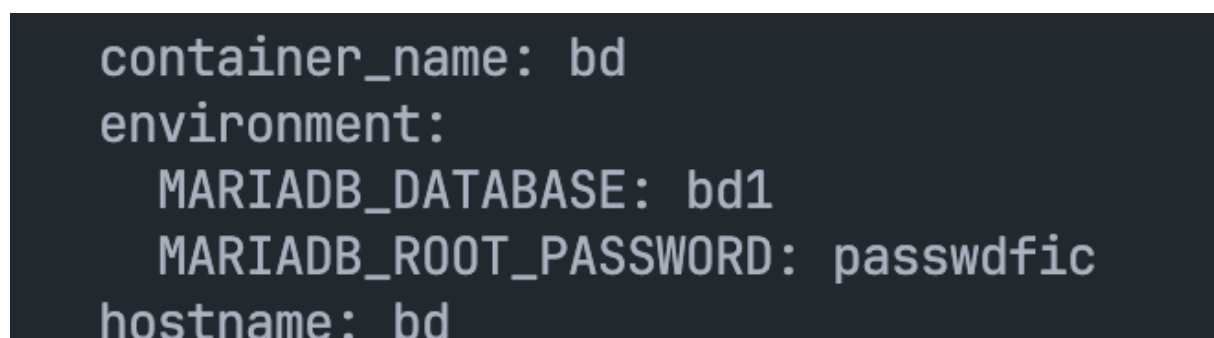
16/



```
compose.yml ServeurWeb  env-bd.yml ServeurWeb
MARIADB_ROOT_PASSWORD=passwdfic
MARIADB_DATABASE=bd1
```

On ajoute simplement une seconde ligne dans le fichier **env-bd.yml** pour définir la variable `MARIADB_DATABASE` avec la valeur `bd1`.

Commande : `docker compose config`



```
container_name: bd
environment:
  MARIADB_DATABASE: bd1
  MARIADB_ROOT_PASSWORD: passwdfic
hostname: bd
```

On vérifie la configuration générée.

Dans la sortie, on doit constater que Docker Compose affiche désormais :

Même si ces variables proviennent du fichier **env-bd.yml**, Docker Compose les **intègre automatiquement** dans la configuration générée, comme si elles étaient définies directement dans `compose.yml`.

17/

Commande : `docker exec -it bd sh`

Commande : `mysql -uroot -p$MARIADB_ROOT_PASSWORD -e "SHOW DATABASES;"`

On affiche les bases existantes dans MariaDB.

On constate que la base **bd1** apparaît aux côtés des bases système (`mysql`, `information_schema`, etc.).

```
tpintegrationcontinue-clairecausse/TPCompose/ServeurWeb on 8 dev [?:]
> docker exec -it bd sh
# mariadb -uroot -p$MARIADB_ROOT_PASSWORD --socket=/run/mysqld/mysqld.sock -e "SHOW DATABASES;"
+-----+
| Database |
+-----+
| bd1      |
| information_schema |
| mysql    |
| performance_schema |
| sys      |
+-----+
```

La variable d'environnement `MARIADB_DATABASE=bd1`, définie dans **env-bd.yml**, demande à MariaDB **de créer automatiquement cette base au premier démarrage** du conteneur.

Comme le conteneur a été relancé après la configuration, MariaDB a créé la base **bd1**, ce qui explique pourquoi elle apparaît dans la liste des bases existantes.

18/

`docker compose down -v`

On arrête tous les services **et** on supprime le volume `donneesbd` grâce à l'option `-v`.

`docker compose up -d`

On relance tous les services, et MariaDB ré-initialise complètement sa base car le volume est vide.

`docker volume ls`

On vérifie que le volume **donneesbd** a bien été recréé automatiquement.

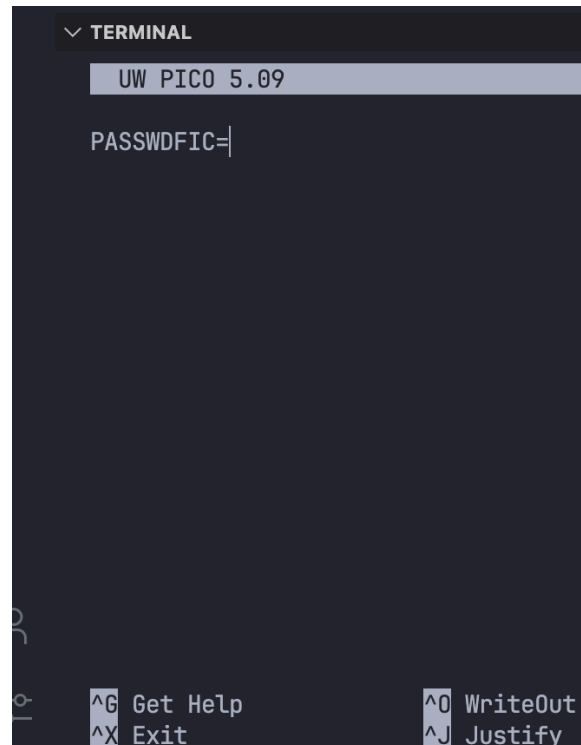
`docker exec -it bd sh`

On entre dans le conteneur MariaDB.

`mariadb -uroot --socket=/run/mysqld/mysqld.sock -e "SHOW DATABASES;"`

On affiche les bases existantes.

19/



```
✓ TERMINAL
UW PICO 5.09
PASSWDFIC=
```

^G Get Help ^O WriteOut  
^X Exit ^J Justify

**Commande :** `docker compose config`

On voit que le mot de passe de MariaDB n'a **pas changé**, même si la variable du fichier `.env` est vide.

**Commande :** `docker exec -it bd sh`

puis : `mariadb -uroot --socket=/run/mysqld/mysqld.sock -e "SHOW DATABASES;"`

On voit que **les mêmes bases existent toujours**, notamment **bd1**.

**Explication :**

MariaDB **n'applique les variables d'environnement que lors du tout premier démarrage**, quand le volume est vide.

Comme le volume existe déjà, **les nouvelles valeurs sont ignorées**, et la base reste identique.

20/

**Commande :** Modifier dans `nginx.conf` la ligne du *root* :

Puis enregistrer le fichier.

**Explication :**

On force volontairement Nginx à utiliser un répertoire **html2** qui n'existe pas, afin que le service **www** passe dans un état d'erreur.

Cela permettra ensuite de tester son état avec les commandes Docker Compose.

21/

**Commande :**

Modifier le service **www** dans `compose.yml` en ajoutant la directive `healthcheck` :

```
www:
  image: nginx
  ports:
    - "80:80"
  depends_on:
    - php
  volumes:
    - ./html:/usr/share/nginx/html
    - ./php:/var/www/html
    - ./nginx-conf/nginx.conf:/etc/nginx/conf.d/default.conf
  networks:
    lemp_compose:
      ipv4_address: 10.20.30.5

  healthcheck:
    test: ["CMD", "curl", "-f", "localhost"]
    interval: 30s
    timeout: 5s
    retries: 3
    start_period: 10s
```

**Explication :**

On ajoute un **healthcheck** pour vérifier si Nginx répond correctement.

- `curl -f localhost` teste si la page renvoie un code HTTP valide.
- `retries: 3` → on teste 3 fois avant de déclarer l'état "unhealthy".
- `interval: 30s` → 30 secondes entre chaque test.
- `timeout: 5s` → si la commande prend plus de 5 secondes, c'est un échec.
- `start_period: 10s` → on attend 10 secondes avant de commencer les tests.

Comme le dossier racine est volontairement incorrect (html2), le conteneur **passera en mauvais état**, ce qui permettra de visualiser son statut.

22/

### Commande :

```
docker compose logs www
```

Cette commande affiche **uniquement les logs du service www**, ce qui permet de voir les erreurs générées par Nginx (par exemple, le répertoire html2 introuvable), et donc de comprendre pourquoi le healthcheck passe en état "unhealthy".